

OA PREPARATION GUIDE

Array DSA Questions & C++ Solutions

50 array-based problems most frequently asked in Online Assessment (OA) rounds — campus placements and product-based company screenings — grouped by pattern, each with approach, working C++ code, and time/space complexity.

COVERAGE

Easy → Hard

LANGUAGE

C++

FORMAT

8 Pattern Sections

Table of Contents

01	Basic Array Operations	12 problems
02	Two-Pointer & Sliding Window	6 problems
03	Sorting-Based & Rearrangement	7 problems
04	Prefix Sum, Kadane & Subarrays	6 problems
05	Binary Search on Arrays	5 problems
06	Hashing-Based Problems	5 problems
07	Matrix / 2D Array Problems	4 problems
08	Advanced / Hard (Product-Based OAs)	5 problems

01 Basic Array Operations

P1. Find the largest element in an array

EASY

Approach: Track the maximum seen so far in a single pass.

```
int findLargest(vector<int>& arr) {
    int maxVal = arr[0];
    for (int i = 1; i < (int)arr.size(); i++)
        if (arr[i] > maxVal) maxVal = arr[i];
    return maxVal;
}
```

Time: $O(n)$ **Space:** $O(1)$

P2. Find the second largest element without sorting

EASY

Approach: Maintain first and second max in one pass; update both when a new max is found.

```
int secondLargest(vector<int>& arr) {
    long first = LONG_MIN, second = LONG_MIN;
    for (int x : arr) {
        if (x > first) { second = first; first = x; }
        else if (x > second && x != first) second = x;
    }
    return second;
}
```

Time: $O(n)$ **Space:** $O(1)$

P3. Reverse an array in place

EASY

Approach: Two pointers from both ends, swap and move inward.

```
void reverseArray(vector<int>& arr) {
    int l = 0, r = arr.size() - 1;
    while (l < r) swap(arr[l++], arr[r--]);
}
```

Time: $O(n)$ **Space:** $O(1)$

P4. Check whether an array is sorted

EASY

Approach: Compare every element with its predecessor; a single violation means it's not sorted.

```
bool isSorted(vector<int>& arr) {
    for (int i = 1; i < (int)arr.size(); i++)
        if (arr[i] < arr[i - 1]) return false;
    return true;
}
```

Time: $O(n)$ **Space:** $O(1)$

P5. Remove duplicates from a sorted array (in-place)

EASY

Approach: Since duplicates are adjacent in a sorted array, keep a write pointer k and only copy an element forward when it differs from the last kept value.

```
int removeDuplicates(vector<int>& arr) {
    if (arr.empty()) return 0;
    int k = 1;
    for (int i = 1; i < (int)arr.size(); i++)
        if (arr[i] != arr[k - 1]) arr[k++] = arr[i];
    return k; // new length
}
```

Time: $O(n)$ **Space:** $O(1)$

P6. Left rotate an array by d positions

EASY

Approach: The classic "reversal algorithm" — reverse the first d elements, reverse the rest, then reverse the whole array. Achieves the rotation in-place without extra arrays.

```
void leftRotate(vector<int>& arr, int d) {
    int n = arr.size();
    d %= n;
    reverse(arr.begin(), arr.begin() + d);
    reverse(arr.begin() + d, arr.end());
    reverse(arr.begin(), arr.end());
}
```

Time: $O(n)$ **Space:** $O(1)$

P7. Move all zeroes to the end (keep relative order)

EASY

Approach: Two-pointer: j tracks the position for the next non-zero element; swap non-zero values into place while scanning with i .

```
void moveZeroes(vector<int>& arr) {
    int j = 0;
    for (int i = 0; i < (int)arr.size(); i++)
        if (arr[i] != 0) swap(arr[i], arr[j++]);
}
```

Time: $O(n)$ **Space:** $O(1)$

P8. Linear search for a key in an unsorted array

EASY

Approach: Scan every element until a match is found.

```
int linearSearch(vector<int>& arr, int key) {
    for (int i = 0; i < (int)arr.size(); i++)
        if (arr[i] == key) return i;
    return -1;
}
```

Time: $O(n)$ **Space:** $O(1)$

P9. Union and intersection of two sorted arrays

MEDIUM

Approach: Merge-style two-pointer walk over both sorted arrays, advancing the smaller pointer and skipping duplicates for union; only advance both together on equality for intersection.

```
vector<int> unionArr(vector<int>& a, vector<int>& b) {
    vector<int> res; int i = 0, j = 0;
    auto push = [&](int v) { if (res.empty() || res.back() != v) res.push_back(v); };
    while (i < (int)a.size() && j < (int)b.size()) {
        if (a[i] < b[j]) push(a[i++]);
        else if (b[j] < a[i]) push(b[j++]);
        else { push(a[i]); i++; j++; }
    }
    while (i < (int)a.size()) push(a[i++]);
    while (j < (int)b.size()) push(b[j++]);
    return res;
}

vector<int> intersection(vector<int>& a, vector<int>& b) {
    vector<int> res; int i = 0, j = 0;
    while (i < (int)a.size() && j < (int)b.size()) {
        if (a[i] < b[j]) i++;
        else if (b[j] < a[i]) j++;
        else { res.push_back(a[i]); i++; j++; }
    }
    return res;
}
```

Time: $O(n+m)$ **Space:** $O(n+m)$ output

P10. Find the missing number in an array of 1..n

EASY

Approach: Use the closed-form sum $n(n+1)/2$ and subtract the actual sum — the difference is the missing value. (XOR of 1..n against all elements also works and avoids overflow.)

```
int missingNumber(vector<int>& arr, int n) {
    long long expected = (long long)n * (n + 1) / 2;
    long long actual = 0;
    for (int x : arr) actual += x;
    return (int)(expected - actual);
}
```

Time: $O(n)$ **Space:** $O(1)$

P11. Maximum consecutive 1s in a binary array

EASY

Approach: Running counter that resets on a 0, tracking the best value seen.

```
int maxConsecutiveOnes(vector<int>& arr) {
    int count = 0, best = 0;
    for (int x : arr) {
        count = (x == 1) ? count + 1 : 0;
        best = max(best, count);
    }
    return best;
}
```

Time: $O(n)$ **Space:** $O(1)$

P12. Find the frequency of every element using hashing

EASY

Approach: A hash map counts occurrences in one pass — avoids the $O(n^2)$ brute force of comparing every pair.

```
unordered_map<int, int> frequency(vector<int>& arr) {
    unordered_map<int, int> freq;
    for (int x : arr) freq[x]++;
    return freq;
}
```

Time: $O(n)$ **Space:** $O(n)$

02 Two-Pointer & Sliding Window

P13. Two Sum — return indices of the pair that adds to target

EASY

Approach: Store each visited value's index in a hash map; for every element check if its complement was already seen.

```
vector<int> twoSum(vector<int>& nums, int target) {
    unordered_map<int, int> seen;
    for (int i = 0; i < (int)nums.size(); i++) {
        int need = target - nums[i];
        if (seen.count(need)) return {seen[need], i};
        seen[nums[i]] = i;
    }
    return {};
}
```

Time: $O(n)$ **Space:** $O(n)$

P14. Pair with a given sum in a sorted array

EASY

Approach: Two pointers from both ends; move left pointer right if the sum is too small, right pointer left if too large.

```
bool pairSumSorted(vector<int>& arr, int target) {
    int l = 0, r = arr.size() - 1;
    while (l < r) {
        int s = arr[l] + arr[r];
        if (s == target) return true;
        else if (s < target) l++;
        else r--;
    }
    return false;
}
```

Time: $O(n)$ **Space:** $O(1)$

P15. Container With Most Water — maximize area between two lines

MEDIUM

Approach: Two pointers starting at both ends; always move the pointer at the shorter line inward, since keeping the taller one can never increase area with a smaller width.

```
int maxArea(vector<int>& height) {
    int l = 0, r = height.size() - 1, best = 0;
    while (l < r) {
        best = max(best, min(height[l], height[r]) * (r - l));
        if (height[l] < height[r]) l++; else r--;
    }
    return best;
}
```

Time: $O(n)$ Space: $O(1)$

P16. Maximum sum subarray of fixed size K

EASY

Approach: Compute the first window's sum, then slide the window by adding the incoming element and removing the outgoing one instead of recomputing the whole sum.

```
int maxSumSubarrayK(vector<int>& arr, int k) {
    int n = arr.size(), windowSum = 0;
    for (int i = 0; i < k; i++) windowSum += arr[i];
    int best = windowSum;
    for (int i = k; i < n; i++) {
        windowSum += arr[i] - arr[i - k];
        best = max(best, windowSum);
    }
    return best;
}
```

Time: $O(n)$ Space: $O(1)$

P17. Smallest subarray with sum $\geq$ target (positive numbers)

MEDIUM

Approach: Variable-size sliding window — expand the right edge to grow the sum, then shrink from the left while the condition still holds, tracking the minimum length.

```
int minSubArrayLen(int target, vector<int>& nums) {
    int n = nums.size(), left = 0, sum = 0, best = INT_MAX;
    for (int right = 0; right < n; right++) {
        sum += nums[right];
        while (sum >= target) {
            best = min(best, right - left + 1);
            sum -= nums[left++];
        }
    }
    return best == INT_MAX ? 0 : best;
}
```

Time: $O(n)$ Space: $O(1)$

P18. 3Sum — all unique triplets that sum to zero**MEDIUM**

Approach: Sort the array, fix one element, then use the two-pointer technique on the remaining range; skip over duplicate values to avoid repeated triplets.

```
vector<vector<int>> threeSum(vector<int> nums) {
    vector<vector<int>> res;
    sort(nums.begin(), nums.end());
    int n = nums.size();
    for (int i = 0; i < n - 2; i++) {
        if (i > 0 && nums[i] == nums[i - 1]) continue;
        int l = i + 1, r = n - 1;
        while (l < r) {
            int sum = nums[i] + nums[l] + nums[r];
            if (sum == 0) {
                res.push_back({nums[i], nums[l], nums[r]});
                while (l < r && nums[l] == nums[l + 1]) l++;
                while (l < r && nums[r] == nums[r - 1]) r--;
                l++; r--;
            } else if (sum < 0) l++;
            else r--;
        }
    }
    return res;
}
```

Time: $O(n^2)$ **Space:** $O(1)$ extra (excluding output)

03 Sorting-Based & Rearrangement

P19. Sort an array of 0s, 1s and 2s (Dutch National Flag)

MEDIUM

Approach: Single pass with three pointers (low, mid, high) that partition the array into 0s, 1s, and 2s regions without a general sort.

```
void sortColors(vector<int>& arr) {
    int low = 0, mid = 0, high = arr.size() - 1;
    while (mid <= high) {
        if (arr[mid] == 0) swap(arr[low++], arr[mid++]);
        else if (arr[mid] == 1) mid++;
        else swap(arr[mid], arr[high--]);
    }
}
```

Time: $O(n)$ **Space:** $O(1)$

P20. Rearrange array into alternating positive and negative numbers

MEDIUM

Approach: Separate elements into positive and negative buckets (order preserved), then interleave them back into the original array.

```
void rearrangePosNeg(vector<int>& arr) {
    vector<int> pos, neg;
    for (int x : arr) (x >= 0 ? pos : neg).push_back(x);
    int i = 0, p = 0, ng = 0;
    while (p < (int)pos.size() && ng < (int)neg.size()) {
        arr[i++] = pos[p++];
        arr[i++] = neg[ng++];
    }
    while (p < (int)pos.size()) arr[i++] = pos[p++];
    while (ng < (int)neg.size()) arr[i++] = neg[ng++];
}
```

Time: $O(n)$ **Space:** $O(n)$

P21. Find all "leaders" in an array (greater than everything to its right)**EASY**

Approach: Scan right to left, keeping the running maximum; an element is a leader exactly when it exceeds that running maximum.

```
vector<int> leaders(vector<int>& arr) {
    vector<int> res;
    int n = arr.size(), maxFromRight = arr[n - 1];
    res.push_back(maxFromRight);
    for (int i = n - 2; i >= 0; i--) {
        if (arr[i] > maxFromRight) {
            maxFromRight = arr[i];
            res.push_back(maxFromRight);
        }
    }
    reverse(res.begin(), res.end());
    return res;
}
```

Time: $O(n)$ **Space:** $O(n)$ output

P22. Next Permutation — rearrange to the lexicographically next greater arrangement**MEDIUM**

Approach: Find the rightmost index i where $\text{nums}[i] < \text{nums}[i+1]$ (the first "descent" from the right). Swap it with the smallest element to its right that's still larger, then reverse the suffix after i to make it ascending (smallest possible).

```
void nextPermutation(vector<int>& nums) {
    int n = nums.size(), i = n - 2;
    while (i >= 0 && nums[i] >= nums[i + 1]) i--;
    if (i >= 0) {
        int j = n - 1;
        while (nums[j] <= nums[i]) j--;
        swap(nums[i], nums[j]);
    }
    reverse(nums.begin() + i + 1, nums.end());
}
```

Time: $O(n)$ **Space:** $O(1)$

P23. Merge two sorted arrays without using extra space

HARD

Approach: The "gap method" (based on Shell sort): start with $\text{gap} = \text{ceil}((n+m)/2)$ and compare elements that many positions apart across both arrays (treated as one logical array), swapping out-of-order pairs; halve the gap each round until it reaches 1.

```
void mergeNoSpace(vector<int>& a, vector<int>& b) {
    int n = a.size(), m = b.size();
    int gap = (n + m + 1) / 2;
    while (gap > 0) {
        int i = 0, j = gap;
        while (j < n + m) {
            int &ai = (i < n) ? a[i] : b[i - n];
            int &aj = (j < n) ? a[j] : b[j - n];
            if (ai > aj) swap(ai, aj);
            i++; j++;
        }
        if (gap == 1) break;
        gap = (gap + 1) / 2;
    }
}
```

Time: $O((n+m) \log(n+m))$ **Space:** $O(1)$

P24. Merge overlapping intervals

MEDIUM

Approach: Sort intervals by start time; then walk through and extend the last merged interval whenever the next one overlaps it, otherwise start a new group.

```
vector<vector<int>> mergeIntervals(vector<vector<int>> intervals) {
    sort(intervals.begin(), intervals.end());
    vector<vector<int>> res;
    for (auto& iv : intervals) {
        if (res.empty() || res.back()[1] < iv[0]) res.push_back(iv);
        else res.back()[1] = max(res.back()[1], iv[1]);
    }
    return res;
}
```

Time: $O(n \log n)$ **Space:** $O(n)$

P25. Minimum number of platforms required for a railway station**MEDIUM**

Approach: Sort arrival and departure times separately; sweep both with two pointers, incrementing a platform counter on an arrival and decrementing on a departure, tracking the peak.

```
int findPlatform(vector<int> arr, vector<int> dep) {
    sort(arr.begin(), arr.end());
    sort(dep.begin(), dep.end());
    int n = arr.size(), platforms = 1, best = 1, i = 1, j = 0;
    while (i < n && j < n) {
        if (arr[i] <= dep[j]) { platforms++; i++; }
        else { platforms--; j++; }
        best = max(best, platforms);
    }
    return best;
}
```

Time: $O(n \log n)$ **Space:** $O(1)$

P26. Kadane's Algorithm — maximum subarray sum**MEDIUM**

Approach: At each index, decide whether to extend the previous subarray or start fresh from the current element — whichever gives a larger sum. Track the best value seen.

```
int maxSubArray(vector<int>& nums) {  
    int best = nums[0], cur = nums[0];  
    for (int i = 1; i < (int)nums.size(); i++) {  
        cur = max(nums[i], cur + nums[i]);  
        best = max(best, cur);  
    }  
    return best;  
}
```

Time: $O(n)$ **Space:** $O(1)$

P27. Maximum product subarray**MEDIUM**

Approach: Track both the running maximum and running minimum product, since a negative number can flip the smallest product into the largest. Swap them whenever the current element is negative.

```
int maxProduct(vector<int>& nums) {  
    int maxP = nums[0], minP = nums[0], best = nums[0];  
    for (int i = 1; i < (int)nums.size(); i++) {  
        if (nums[i] < 0) swap(maxP, minP);  
        maxP = max(nums[i], maxP * nums[i]);  
        minP = min(nums[i], minP * nums[i]);  
        best = max(best, maxP);  
    }  
    return best;  
}
```

Time: $O(n)$ **Space:** $O(1)$

P28. Count subarrays with sum exactly K

MEDIUM

Approach: Maintain a running prefix sum and a hash map of how many times each prefix sum has occurred. If $\text{sum} - k$ was seen before, every one of those occurrences marks the start of a valid subarray ending here.

```
int subarraySum(vector<int>& nums, int k) {
    unordered_map<int, int> prefixCount;
    prefixCount[0] = 1;
    int sum = 0, count = 0;
    for (int x : nums) {
        sum += x;
        if (prefixCount.count(sum - k)) count += prefixCount[sum - k];
        prefixCount[sum]++;
    }
    return count;
}
```

Time: $O(n)$ **Space:** $O(n)$

P29. Equilibrium / pivot index (left sum equals right sum)

EASY

Approach: Compute the total sum once; then walk left to right maintaining `leftSum`, checking whether it equals $\text{total} - \text{leftSum} - \text{nums}[i]$ (the right side).

```
int pivotIndex(vector<int>& nums) {
    int total = accumulate(nums.begin(), nums.end(), 0);
    int leftSum = 0;
    for (int i = 0; i < (int)nums.size(); i++) {
        if (leftSum == total - leftSum - nums[i]) return i;
        leftSum += nums[i];
    }
    return -1;
}
```

Time: $O(n)$ **Space:** $O(1)$

P30. Product of array except self (no division allowed)

MEDIUM

Approach: Build a prefix-product array from the left, then sweep from the right multiplying in a running suffix product — avoids division entirely.

```
vector<int> productExceptSelf(vector<int>& nums) {
    int n = nums.size();
    vector<int> res(n, 1);
    for (int i = 1; i < n; i++) res[i] = res[i - 1] * nums[i - 1];
    int right = 1;
    for (int i = n - 1; i >= 0; i--) {
        res[i] *= right;
        right *= nums[i];
    }
    return res;
}
```

Time: $O(n)$ **Space:** $O(1)$ extra (excluding output)

P31. Longest consecutive sequence in an unsorted array

MEDIUM

Approach: Put all elements in a hash set. Only start counting a sequence from a number that has no predecessor ($x-1$ not in the set), then extend forward — this guarantees each sequence is counted exactly once, giving $O(n)$ overall instead of sorting.

```
int longestConsecutive(vector<int>& nums) {
    unordered_set<int> s(nums.begin(), nums.end());
    int best = 0;
    for (int x : s) {
        if (!s.count(x - 1)) {
            int len = 1;
            while (s.count(x + len)) len++;
            best = max(best, len);
        }
    }
    return best;
}
```

Time: $O(n)$ **Space:** $O(n)$

05 Binary Search on Arrays

P32. Classic binary search on a sorted array

EASY

Approach: Repeatedly halve the search range by comparing the middle element to the key.

```
int binarySearch(vector<int>& arr, int key) {
    int lo = 0, hi = arr.size() - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (arr[mid] == key) return mid;
        else if (arr[mid] < key) lo = mid + 1;
        else hi = mid - 1;
    }
    return -1;
}
```

Time: $O(\log n)$ **Space:** $O(1)$

P33. Search in a rotated sorted array

MEDIUM

Approach: At each step, at least one half (left-of-mid or mid-to-right) is guaranteed to be normally sorted. Determine which half is sorted, check if the target lies in that half's range, and recurse into the correct half.

```
int searchRotated(vector<int>& arr, int target) {
    int lo = 0, hi = arr.size() - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (arr[mid] == target) return mid;
        if (arr[lo] <= arr[mid]) {
            if (arr[lo] <= target && target < arr[mid]) hi = mid - 1;
            else lo = mid + 1;
        } else {
            if (arr[mid] < target && target <= arr[hi]) lo = mid + 1;
            else hi = mid - 1;
        }
    }
    return -1;
}
```

Time: $O(\log n)$ **Space:** $O(1)$

P34. Find a peak element (greater than both neighbors)

MEDIUM

Approach: Binary search on the "slope": if $\text{nums}[\text{mid}] > \text{nums}[\text{mid}+1]$, a peak must exist on the left side (including mid), otherwise it's on the right.

```
int findPeakElement(vector<int>& nums) {
    int lo = 0, hi = nums.size() - 1;
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (nums[mid] > nums[mid + 1]) hi = mid;
        else lo = mid + 1;
    }
    return lo;
}
```

Time: $O(\log n)$ **Space:** $O(1)$

P35. Find the minimum element in a rotated sorted array

MEDIUM

Approach: Binary search comparing $\text{nums}[\text{mid}]$ with $\text{nums}[\text{hi}]$: if $\text{nums}[\text{mid}] > \text{nums}[\text{hi}]$, the minimum is to the right of mid; otherwise it's at mid or to its left.

```
int findMin(vector<int>& nums) {
    int lo = 0, hi = nums.size() - 1;
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (nums[mid] > nums[hi]) lo = mid + 1;
        else hi = mid;
    }
    return nums[lo];
}
```

Time: $O(\log n)$ **Space:** $O(1)$

P36. Median of two sorted arrays

HARD

Approach: Binary search on the smaller array to find a partition point such that all elements on the left of the combined partition are $\leq$ all elements on the right, in $O(\log(\min(n,m)))$ instead of merging.

```
double findMedianSortedArrays(vector<int> a, vector<int> b) {
    if (a.size() > b.size()) swap(a, b);
    int n = a.size(), m = b.size();
    int lo = 0, hi = n;
    while (lo <= hi) {
        int cutA = (lo + hi) / 2;
        int cutB = (n + m + 1) / 2 - cutA;
        int leftA = (cutA == 0) ? INT_MIN : a[cutA - 1];
        int leftB = (cutB == 0) ? INT_MIN : b[cutB - 1];
        int rightA = (cutA == n) ? INT_MAX : a[cutA];
        int rightB = (cutB == m) ? INT_MAX : b[cutB];
        if (leftA <= rightB && leftB <= rightA) {
            if ((n + m) % 2 == 0) return (max(leftA, leftB) + min(rightA, rightB)) / 2.0;
            return max(leftA, leftB);
        } else if (leftA > rightB) hi = cutA - 1;
        else lo = cutA + 1;
    }
    return 0.0;
}
```

Time: $O(\log(\min(n,m)))$ **Space:** $O(1)$

P37. Find all duplicates in an array where $1 \leq a[i] \leq n$ **MEDIUM**

Approach: Use each value as an index and negate the element at that index. If you land on an already-negative value, that index+1 is a duplicate — achieves $O(1)$ extra space by reusing the array itself.

```
vector<int> findDuplicates(vector<int>& nums) {
    vector<int> res;
    for (int i = 0; i < (int)nums.size(); i++) {
        int idx = abs(nums[i]) - 1;
        if (nums[idx] < 0) res.push_back(idx + 1);
        else nums[idx] = -nums[idx];
    }
    return res;
}
```

Time: $O(n)$ **Space:** $O(1)$ extra (excluding output)

P38. Find the majority element (appears $> n/2$ times)**MEDIUM**

Approach: Boyer-Moore Voting: maintain a candidate and a count; increment on a match, decrement otherwise, and switch candidates when the count hits zero. Guaranteed correct because a true majority element can't be fully cancelled out.

```
int majorityElement(vector<int>& nums) {
    int count = 0, candidate = 0;
    for (int x : nums) {
        if (count == 0) candidate = x;
        count += (x == candidate) ? 1 : -1;
    }
    return candidate;
}
```

Time: $O(n)$ **Space:** $O(1)$

P39. Find the single non-repeating element (every other element appears twice)**EASY**

Approach: XOR every element together — identical pairs cancel out to zero, leaving only the unique value.

```
int singleNumber(vector<int>& nums) {
    int result = 0;
    for (int x : nums) result ^= x;
    return result;
}
```

Time: $O(n)$ **Space:** $O(1)$

P40. First non-repeating element in an array**EASY**

Approach: Count frequencies with a hash map in one pass, then scan the array again (to preserve original order) and return the first element with frequency 1.

```
int firstUnique(vector<int>& arr) {
    unordered_map<int, int> freq;
    for (int x : arr) freq[x]++;
    for (int x : arr) if (freq[x] == 1) return x;
    return -1;
}
```

Time: $O(n)$ **Space:** $O(n)$

P41. Count inversions in an array (pairs where $i < j$ but $a[i] > a[j]$)**HARD**

Approach: Modified merge sort — while merging two sorted halves, whenever an element from the right half is placed before elements remaining in the left half, all of those remaining left elements form inversions with it.

```
long long mergeCount(vector<int>& arr, int l, int mid, int r) {
    vector<int> temp;
    int i = l, j = mid + 1;
    long long inv = 0;
    while (i <= mid && j <= r) {
        if (arr[i] <= arr[j]) temp.push_back(arr[i++]);
        else { inv += (mid - i + 1); temp.push_back(arr[j++]); }
    }
    while (i <= mid) temp.push_back(arr[i++]);
    while (j <= r) temp.push_back(arr[j++]);
    for (int k = l; k <= r; k++) arr[k] = temp[k - l];
    return inv;
}

long long countInversions(vector<int>& arr, int l, int r) {
    if (l >= r) return 0;
    int mid = l + (r - l) / 2;
    long long inv = countInversions(arr, l, mid) + countInversions(arr, mid + 1, r);
    return inv + mergeCount(arr, l, mid, r);
}
```

Time: $O(n \log n)$ **Space:** $O(n)$

07 Matrix / 2D Array Problems

P42. Rotate a square matrix by 90° in place

MEDIUM

Approach: Transpose the matrix (swap $\text{mat}[i][j]$ with $\text{mat}[j][i]$), then reverse every row — equivalent to a clockwise 90° rotation without extra space.

```
void rotate(vector<vector<int>>& mat) {
    int n = mat.size();
    for (int i = 0; i < n; i++)
        for (int j = i + 1; j < n; j++)
            swap(mat[i][j], mat[j][i]);
    for (auto& row : mat) reverse(row.begin(), row.end());
}
```

Time: $O(n^2)$ **Space:** $O(1)$

P43. Print a matrix in spiral order

MEDIUM

Approach: Maintain four shrinking boundaries (top, bottom, left, right) and traverse right → down → left → up, tightening the boundary after each side.

```
vector<int> spiralOrder(vector<vector<int>>& mat) {
    vector<int> res;
    if (mat.empty()) return res;
    int top = 0, bottom = mat.size() - 1, left = 0, right = mat[0].size() - 1;
    while (top <= bottom && left <= right) {
        for (int j = left; j <= right; j++) res.push_back(mat[top][j]);
        top++;
        for (int i = top; i <= bottom; i++) res.push_back(mat[i][right]);
        right--;
        if (top <= bottom) {
            for (int j = right; j >= left; j--) res.push_back(mat[bottom][j]);
            bottom--;
        }
        if (left <= right) {
            for (int i = bottom; i >= top; i--) res.push_back(mat[i][left]);
            left++;
        }
    }
    return res;
}
```

Time: $O(n \cdot m)$ **Space:** $O(1)$ extra (excluding output)

P44. Set entire row and column to zero if an element is zero**MEDIUM**

Approach: Use the matrix's first row and first column themselves as marker flags for which rows/columns must be zeroed, tracking whether the first row/column originally needed zeroing separately — avoids $O(n+m)$ extra marker arrays.

```
void setZeroes(vector<vector<int>>& mat) {
    int m = mat.size(), n = mat[0].size();
    bool firstRow = false, firstCol = false;
    for (int j = 0; j < n; j++) if (mat[0][j] == 0) firstRow = true;
    for (int i = 0; i < m; i++) if (mat[i][0] == 0) firstCol = true;
    for (int i = 1; i < m; i++)
        for (int j = 1; j < n; j++)
            if (mat[i][j] == 0) { mat[i][0] = 0; mat[0][j] = 0; }
    for (int i = 1; i < m; i++)
        for (int j = 1; j < n; j++)
            if (mat[i][0] == 0 || mat[0][j] == 0) mat[i][j] = 0;
    if (firstRow) for (int j = 0; j < n; j++) mat[0][j] = 0;
    if (firstCol) for (int i = 0; i < m; i++) mat[i][0] = 0;
}
```

Time: $O(n \cdot m)$ **Space:** $O(1)$

P45. Search a target in a row-wise and column-wise sorted matrix**MEDIUM**

Approach: Start at the top-right corner. If the current value is too big, move left (eliminates a column); if too small, move down (eliminates a row) — a "staircase" search.

```
bool searchMatrix(vector<vector<int>>& mat, int target) {
    int m = mat.size(), n = mat[0].size();
    int i = 0, j = n - 1;
    while (i < m && j >= 0) {
        if (mat[i][j] == target) return true;
        else if (mat[i][j] > target) j--;
        else i++;
    }
    return false;
}
```

Time: $O(n+m)$ **Space:** $O(1)$

P46. Trapping Rain Water

HARD

Approach: Two pointers from both ends tracking leftMax/rightMax. Always process the side with the smaller max — water trapped at that position is bounded by that side's max minus its height.

```
int trap(vector<int>& height) {
    int l = 0, r = height.size() - 1, leftMax = 0, rightMax = 0, water = 0;
    while (l < r) {
        if (height[l] < height[r]) {
            if (height[l] >= leftMax) leftMax = height[l];
            else water += leftMax - height[l];
            l++;
        } else {
            if (height[r] >= rightMax) rightMax = height[r];
            else water += rightMax - height[r];
            r--;
        }
    }
    return water;
}
```

Time: $O(n)$ Space: $O(1)$

P47. Best Time to Buy and Sell Stock (one transaction)

EASY

Approach: Track the minimum price seen so far while scanning; at every day compute the profit if sold today and keep the best.

```
int maxProfit(vector<int>& prices) {
    int minPrice = INT_MAX, best = 0;
    for (int p : prices) {
        minPrice = min(minPrice, p);
        best = max(best, p - minPrice);
    }
    return best;
}
```

Time: $O(n)$ Space: $O(1)$

P48. Best Time to Buy and Sell Stock II (unlimited transactions)**MEDIUM**

Approach: Greedily capture every positive day-to-day price increase — summing all "upward" differences equals the maximum achievable profit with unlimited transactions.

```
int maxProfitMany(vector<int>& prices) {
    int profit = 0;
    for (int i = 1; i < (int)prices.size(); i++)
        if (prices[i] > prices[i - 1]) profit += prices[i] - prices[i - 1];
    return profit;
}
```

Time: $O(n)$ **Space:** $O(1)$

P49. Find the Kth largest element in an unsorted array**MEDIUM**

Approach: Quickselect — a partition-based selection algorithm (like quicksort's partition step) that only recurses into the half containing the target rank, giving average $O(n)$ instead of a full $O(n \log n)$ sort.

```
int quickSelect(vector<int>& nums, int lo, int hi, int k) {
    int pivot = nums[hi], i = lo;
    for (int j = lo; j < hi; j++)
        if (nums[j] >= pivot) swap(nums[i++], nums[j]);
    swap(nums[i], nums[hi]);
    if (i == k) return nums[i];
    return (i < k) ? quickSelect(nums, i + 1, hi, k) : quickSelect(nums, lo, i - 1, k);
}

int findKthLargest(vector<int>& nums, int k) {
    return quickSelect(nums, 0, nums.size() - 1, k - 1);
}
```

Time: $O(n)$ average, $O(n^2)$ worst case **Space:** $O(1)$

P50. Maximum subarray sum in a circular array**HARD**

Approach: The answer is either a normal (non-wrapping) Kadane's max subarray, or a wrapping one equal to total sum - minimum subarray sum. Compute both with a single pass and take the larger (guard against the all-negative case, where the wrapping formula degenerates to an empty subarray).

```
int maxSubarraySumCircular(vector<int>& nums) {
    int total = 0, curMax = 0, maxSum = INT_MIN, curMin = 0, minSum = INT_MAX;
    for (int x : nums) {
        curMax = max(curMax + x, x); maxSum = max(maxSum, curMax);
        curMin = min(curMin + x, x); minSum = min(minSum, curMin);
        total += x;
    }
    if (maxSum < 0) return maxSum; // all negative
    return max(maxSum, total - minSum);
}
```

Time: $O(n)$ **Space:** $O(1)$